



SHASU VATHANAN

SHASU VATHANAN - GEN AI - PRODUCT MANAGER

GEN AI ARCHITECT JOURNEY · PHASE-003

Robotic Process Automation

Driving the desktop and the browser from Python

Phase	Phase-003 - Automation
Folder	Phase-003-Robotic-Process-Automation-(RPA)
Tracks	PyAutoGUI (desktop) · Playwright (browser)
Projects	Six - two fundamentals sets and four complete bots
Level	Intermediate
Platform	Windows 10 or 11 · Python 3.10+

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · [linkedin.com/in/shasuvathanan](https://www.linkedin.com/in/shasuvathanan)



Contents

This phase covers the two ways software can operate other software: by imitating a person at the keyboard, or by speaking to an application through its own automation interface. Each has a fundamentals set and two complete bots.

1	The two approaches
2	PyAutoGUI - driving the desktop
3	Playwright - driving the browser
4	Requirements and setup
5	Safety and responsible use
6	What is committed, and what is not
7	Known limitations

1. The two approaches

The choice is not about which tool is better. It is about what you are automating.

	PyAutoGUI	Playwright
Works by	Moving the physical mouse and pressing real keys	Calling the browser's own automation API
Best for	Desktop programs such as Excel and Notepad	Websites and browser applications
Finds things by	Screen coordinates and pixels	Page elements and selectors
Sensitive to	Screen focus, timing, resolution, window position	Page structure changes only
Needs the desktop	Unlocked and untouched for the whole run	Not at all - it can run headless
Reliability	Lower - the screen is a moving target	Higher - selectors survive a resize

The practical rule

If the target is a web page, use **Playwright** - selectors survive a change of screen resolution and a moved window; pixel coordinates do not. Reach for **PyAutoGUI** only when there is no API at all, which in practice means a desktop application.

2. PyAutoGUI - driving the desktop

PyAutoGUI simulates physical mouse and keyboard actions, so it can work with browsers and native Windows applications that expose no automation API at all. That generality is also its weakness: it sends input to whichever window happens to have focus.

Fundamentals



Script	What it demonstrates
basic_pyautogui.py	The upper-left-corner fail-safe and a one-second pause after each action
pyautogui_mouse_operation.py	Move, click, double-click, right-click, and middle-click
pyautogui_keyboard_operation.py	Typing into the focused window, plus select, copy, Enter, and paste
pyautogui_screenshot_operation.py	Capturing the full screen to screenshot.png

Bengaluru Weather Bot

Opens AccuWeather in Chrome, copies the visible page text, extracts the current conditions, pastes a formatted report into Notepad, and saves it under a date-based filename.

FLOW

```
Launch Chrome -> open AccuWeather -> copy page text -> parse readings  
-> open Notepad -> paste formatted data -> save YYYY-MM-DD_Weather_Data.txt
```

The report can carry temperature, RealFeel, condition, wind, gusts, humidity, indoor humidity, dew point, pressure, cloud cover, visibility, cloud ceiling, capture time, and source URL. When a value cannot be identified the script records **Not found** rather than guessing or failing.

Where the real difficulty sits

Copying the page is trivial. Identifying *which* of the copied lines are the current readings is the actual work - which is why the parser is built around labelled regular expressions rather than fixed line offsets that would break the first time the page reflowed.

Apple Stock Price Bot

Opens the Apple (AAPL) quote on Google Finance, extracts the current price, drives Microsoft Excel through keyboard shortcuts to build a formatted row, saves a dated workbook, and captures a screenshot of the finished sheet.

Column	Contents
Date & Time	When the quote was captured
Stock Company Name	Apple Inc.
Stock Data	The extracted USD price
Source link	The Google Finance URL
Comment	Run notes

A dated report already present is never overwritten - the bot appends the current time to the new filenames instead. **python daily_report_bot.py --check** runs a preflight that verifies the applications and output directory without opening the website or taking over the desktop.



3. Playwright - driving the browser

Playwright locates page elements through the browser's document structure and can wait for navigation and dynamic content. Nothing depends on where a window sits on screen.

Fundamentals

Script	What it demonstrates
Playwright_basic.py	Opens Google in headless Chromium, saves google.png, closes the browser
Playwright_Functionality.py	A commented reference of common Playwright capabilities

Concepts covered across the two projects: starting and closing browsers and contexts, headed and headless execution, navigation and URL waits, locating elements by stable attributes, filling inputs, extracting text, full-page screenshots, reusing a persistent profile, handling timeouts and selector fallbacks, and producing JSON and Excel reports.

SauceDemo Product Automation

Signs in to the SauceDemo training store, visits every product page, extracts the product details, captures one screenshot per product, and writes a formatted Excel report.

- Selects elements by SauceDemo's stable **data-test** attributes rather than by position.
- Overlays the product URL onto the page before each screenshot, because browser screenshots do not include the address bar.
- Builds Windows-safe, unique screenshot filenames.
- Saves partial results and per-product errors, so one failure never costs the whole report.
- Formats and freezes the Excel header, adds a filter, sizes columns, and makes the URLs clickable.

WhatsApp Automation Bot

A consent-based messaging workflow in Microsoft Edge: it sends a personalised message, monitors for a reply, sends one conditional follow-up, captures the incoming messages, and generates dated JSON and Excel reports.

Time	What the bot does
0s	Sends the personalised opening message
0-300s	Watches for a new incoming reply
180s	Sends exactly one follow-up, only if no reply has arrived
on reply	Sends a confirmation immediately
300s	Records no_reply and moves to the next contact

Consent is a design constraint, not a footnote

This bot is for people who have explicitly agreed to receive the messages. Automated or bulk messaging may violate WhatsApp's terms and can get an account restricted. Phone numbers are masked in every log and report the bot produces - the generated report shows **+91*****30**, never the full number.



4. Requirements and setup

- Windows 10 or Windows 11 for the desktop-automation projects
- Python 3.10 or newer
- Internet access
- Google Chrome for the PyAutoGUI web-data bots
- Microsoft Excel for the stock-price bot
- Microsoft Edge and a WhatsApp account for the messaging bot

Each project carries its own setup and run instructions. The common starting point is always the same:

```
POWERSHELL
cd "path\to\the\selected\project"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Then install the dependencies listed in that project's own README. Create a fresh environment inside the project you want to run - virtual environments are machine-specific and are never committed.

5. Safety and responsible use

- PyAutoGUI scripts take control of the active desktop. Save other work, keep the desktop unlocked, and do not touch the mouse or keyboard during a run.
- The fail-safe is enabled in the complete desktop bots - move the pointer to the upper-left corner to stop the automation immediately.
- Test with non-sensitive data before pointing any bot at real accounts or documents.
- Use the messaging bot only with contacts who have explicitly consented.
- Treat contact spreadsheets, browser profiles, conversation reports, and screenshots as private data. Do not publish them without review and redaction.
- Stock quotes may be delayed. These are educational projects, not financial advice or a trading system.

6. What is committed, and what is not

A portfolio repository should carry the source and enough evidence to show the work runs - and nothing else.

Committed	Deliberately excluded
Every script and its README	Screen recordings - 80 to 100 MB each, above what GitHub accepts
Generated Excel reports and product screenshots	contacts.xlsx - holds real, unmasked phone numbers
Sample dated weather reports	The WhatsApp browser profile - a live logged-in session
Requirements files and .gitignore rules	Generated conversation reports and their screenshots
The project briefs and requirements notes	Virtual environments and __pycache__ - machine-specific



7. Known limitations

Websites and desktop interfaces change over time. Page selectors, copied page text, application shortcuts, or screen coordinates may eventually need adjustment. Display scaling, pop-ups, cookie dialogs, slow connections, and window focus can all affect desktop automation.

This is the honest trade-off of GUI automation, and the reason the next phase moves to interfaces designed to be called: a Streamlit app and a FastAPI service have no screen to lose focus and no layout to shift underneath them.

Next

Phase-004-FastAPI-And-Streamlit turns this work into something other people can use - web interfaces with Streamlit and web APIs with FastAPI.

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · linkedin.com/in/shasuvathanan